

开发单客户端网络应用

北京理工大学计算机学院
金旭亮



概述



与其他平台或语言的网络库不太一样，Java将Socket明确地区分为客户端与服务端两种。



与客户端Socket不同，服务端Socket需要监听一个特定的端口，当接收到客户端连接请求时，需要在独立的线程中响应并处理客户端请求。



Server端Socket与Client端Socket相互通讯时，需要遵循事先约定好的一些规则，否则，双方无法顺利地交换数据。

ServerSocket类

用于开发服务端应用

构造函数:

- `ServerSocket()`
- `ServerSocket(int port)`
- `ServerSocket(int port, int backlog)`
- `ServerSocket(int port, int backlog, InetAddress bindAddr)`



`backlog`用于设定连接请求的等待队列长度。

服务端Socket的连接队列



默认情况下，Java将这个连接队列的长度设置为50。

ServerSocket示例

```
public class DayTimeServer {  
    2 usages  
    public final static int PORT = 8899;  
    public static void main(String[] args) {  
        try (ServerSocket server = new ServerSocket(PORT)) {  
            System.out.println("正在监听端口" + PORT + ",等待客户端连接.....");  
            while (true) {  
                //数据发送完毕之后,自动地断开与Client端的连接  
                try (Socket clientConnection = server.accept()) {  
                    System.out.println("接收到客户端连接: " + clientConnection);  
                    //向客户端发送数据,数据以"\r\n"结束  
                    var out = new BufferedWriter(new OutputStreamWriter(  
                        clientConnection.getOutputStream(),  
                        StandardCharsets.UTF_8));  
                    Date now = new Date();  
                    out.write(now.toString());  
                    out.newLine();  
                    out.flush(); //通知底层操作系统,将缓冲区的数据全部发送出去  
                } catch (IOException e) {  
                }  
            }  
        } catch (IOException e) {  
            System.out.println(e);  
        }  
    }  
}
```

创建ServerSocket时,需要指定其监听的端口。对象构建完毕之后,调用其accept()方法它就在指定的端口处监听客户端的连接请求。

accept()方法是一个阻塞调用,当它收到一个连接请求时,会创建一个新的Socket对象,Server端网络应用程序就可以使用此Socket与客户端通信。

服务端与客户端的通信,是通过“**流(stream)**”来实现的。

网络编程中常用的Stream类

缓存流

字符流

读:

```
BufferedReader-->InputStreamReader  
-->socket.getInputStream()
```

字节流

写:

```
BufferedWriter-->OutputStreamWriter  
-->socket.getOutputStream()
```

客户端接收服务端发回的时间信息

```
// 从Server端获取当前的时间
// 只接收数据, 不发送数据
public static void main(String[] args) {
    String server = "127.0.0.1";
    int port = 8899;
    //Socket创建之后, 自动发起连接
    try (Socket socket = new Socket(server, port)) {
        Reader reader = new InputStreamReader(socket.getInputStream(),
            StandardCharsets.UTF_8);
        //从输入流中读取一行 (因为Server端的数据是以"\r\n"结尾的)
        BufferedReader bufferedReader = new BufferedReader(reader);
        String now = bufferedReader.readLine();
        if (now != null) {
            System.out.println(now);
        } else {
            System.out.println("未能成功地读入数据。");
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

DataTimeClient.java

客户端与服务端相互交换数据时，一定要事先约定好相应的通信规则。

在本例中，通信规则如下：

- (1) 客户端只向服务端发出连接请求
- (2) 连接建立之后，服务端将当前时间转换为字符串，以“\r\n”结尾，发给客户端。
- (3) 客户端以“\r\n”为依据按行接收数据。
- (4) 数据传输完毕之后，客户端与服务端的Socket都被断开。

示例分析



在前页PPT所展示的例子中，Server端程序是在监听线程中完成所有数据传输工作的，这就意味着它一次只能为一个客户端服务。



当Server端程序正在为一个客户端服务时，如果有新的连接请求，它不会响应，直到处理完当前的工作之后，才能响应下一个连接请求。



如果单个连接的处理时间过长，将导致服务端无法及时响应外部新的连接请求，因此，这种最简单的模式仅适合于并发连接数很少，并且响应每个连接要完成的工作非常少，处理得非常快这种情形。

下一讲，我们介绍如何引入多线程来处理多客户端同时发起连接的应用场景.....